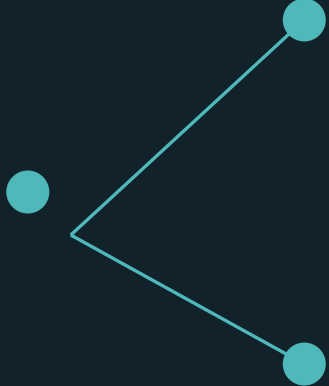




PROJETO DE PARADIGMAS DE PROGRAMAÇÃO · PROJETO FINAL

DSL WebNavi

Uma linguagem de domínio para modelar, validar e gerar testes de fluxos de navegação web — implementada como DSL interna em Racket/Scheme, via macros.

Thiago Dorta
Gabriel Pereira



O problema que a WebNavi resolve

1

Contextualização

Fluxos de navegação web (login, checkout, cadastro) são acordados entre produto e desenvolvimento, mas raramente têm uma fonte única e formal de verdade.

2

Motivação

Ferramentas de teste como Cypress exigem JavaScript. Quem define a regra de negócio não escreve — nem lê — o teste que a valida.

3

Relevância

Uma DSL declarativa em português, validada estaticamente e capaz de gerar testes executáveis, elimina a distância entre especificação e verificação.

Páginas e elementos

Toda construção da WebNavi é uma s-expression Racket. Páginas declaram rotas e elementos interativos; elementos recebem modificadores que descrevem validação e apresentação.

```
(pagina login
  (rota "/login")
  (elemento campo_email entrada
    obrigatorio formato email)
  (elemento campo_senha entrada
    obrigatorio formato senha)
  (elemento botao_entrar botao texto "Entrar"))
```

TIPOS DE ELEMENTO

botao · entrada · link · formulario · modal · aviso · selecao · caixa

FORMATOS DE ENTRADA

email · senha · cpf · cnpj · telefone · cep · data

MODIFICADORES

obrigatorio · minimo N · maximo N · texto "..." · seletor "..." · visivel quando X

Transições, guardas e efeitos

```
(transicao de login para catalogo
  (via botao_entrar clicado)
  (somente se
    (e (preenchido campo_email)
        (preenchido campo_senha))))
  (entao
    (recebe sessao.autenticado verdadeiro)))
```

via

elemento que disparou + tipo de evento (clicado, enviado, selecionado)

somente se

condição (guarda) que deve ser satisfeita para a transição ocorrer

entao

lista de efeitos: valores atribuídos a variáveis de estado do site

Invariantes, dados e fluxos

```
(invariante checkout exige autenticacao  
  (sempre que (estiver em checkout)  
    entao (igual sessao.autenticado a verdadeiro)))
```

```
(dados usuario_valido  
  (email "user@loja.com") (senha "Senha123"))
```

Invariantes expressam propriedades que devem valer sempre que uma página é alcançada; datasets fornecem fixtures reutilizáveis; fluxos descrevem sequências de passos do usuário.

```
(fluxo compra completa  
  (comecar em inicio)  
  (passo clicar botao_login  
    aguardar pagina login)  
  (passo preencher campo_email  
    com usuario_valido.email)  
  (terminar em confirmacao))
```

EBNF — formalismo adotado

Gramática descrita em EBNF (a verificação sintática é feita pelo próprio compilador Racket, via macros `syntax-rules`).

```
programa      ::= (webnavi IDENT declaracao*)
declaracao    ::= pagina | inicio | final | transicao | invariante | dados | fluxo

pagina        ::= (pagina IDENT rota? elemento*)
rota          ::= (rota STRING)
elemento      ::= (elemento IDENT tipo_elem modificador*)
tipo_elem     ::= botao | entrada | link | formulario | modal | aviso | selecao | caixa
modificador   ::= obrigatorio
                | formato tipo_fmt
                | minimo NUMERO
                | maximo NUMERO
                | texto STRING
                | seletor STRING
                | visivel quando IDENT
tipo_fmt      ::= email | senha | cpf | cnpj | telefone | cep | data

inicio        ::= (inicio em IDENT)
final         ::= (final em IDENT*)

transicao      ::= (transicao de IDENT para IDENT via somente? entao?)
via           ::= (via IDENT evento)
evento        ::= clicado | enviado | selecionado
somente       ::= (somente se expressao)
entao         ::= (entao efeito*)
efeito        ::= (recebe ACESSO valor)
valor         ::= STRING | NUMERO | verdadeiro | falso | IDENT | ACESSO

invariante    ::= (invariante IDENT+ (sempre que expressao entao expressao))
```

EBNF — formalismo adotado

```
dados      ::= (dados IDENT campo*)
campo      ::= (IDENT valor)

fluxo      ::= (fluxo IDENT+ (comecar em IDENT) passo+ (terminar em IDENT))
passo      ::= (passo clicar IDENT)
           | (passo clicar IDENT aguardar pagina IDENT)
           | (passo clicar IDENT aguardar IDENT visivel)
           | (passo preencher IDENT com REF_DADOS)
           | (passo navegar IDENT)
           | (passo navegar IDENT aguardar pagina IDENT)

REF_DADOS  ::= IDENT.IDENT | STRING

expressao  ::= (e expressao+)
           | (ou expressao+)
           | (nao expressao)
           | (preenchido IDENT)
           | (vazio IDENT)
           | (visivel IDENT)
           | (oculto IDENT)
           | (estiver em IDENT)
           | (maior ACESSO que NUMERO)
           | (menor ACESSO que NUMERO)
           | (igual ACESSO a valor)
           | (diferente ACESSO de valor)

IDENT      ::= símbolo Scheme, por exemplo login, campo_email, sessao.autenticado
STRING     ::= texto entre aspas
NUMERO     ::= inteiro
ACESSO     ::= símbolo com ponto, por exemplo sessao.autenticado
```

Macros — o compilador é o parser

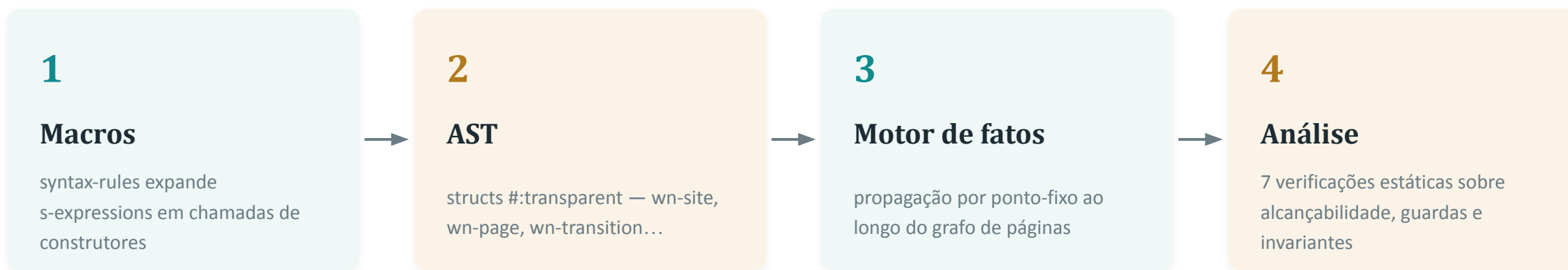
```
(define-syntax transicao
  (syntax-rules (de para)
    [(_ de origem para destino clausula ...)
     (build-transition 'origem 'destino
                       (list clausula ...))]))
```

- **de** e **para** são literais do padrão — exigidos textualmente, não variáveis.
- Qualquer outro símbolo nessas posições é erro de sintaxe na expansão.
- **syntax-rules** seleciona o padrão certo pela estrutura da forma — despacho sintático, antes do programa rodar.

14 macros DSL no total:

webnavi · pagina · rota · elemento · inicio · final · transicao ·
via · somente · entao · recebe · invariante · dados · fluxo · passo

Da macro à verificação: a arquitetura



As macros não geram código diretamente, elas constroem uma AST. Isso separa sintaxe de semântica e permite múltiplos backends (análise, geração de testes) sobre o mesmo modelo.

O usuário escreve um modelo WebNavi utilizando a sintaxe da DSL. As macros implementadas em Scheme transformam esse modelo em uma AST composta por páginas, transições, invariantes e fluxos. Sobre essa AST são executadas análises semânticas e geradores de artefatos, como a geração automática de testes Cypress.

Caso de uso: loja virtual completa

(webnavi loja

(pagina inicio

(rota "/"

(elemento botao_login botao texto "Entrar")

(elemento link_produtos link texto "Ver produtos"))

(pagina login

(rota "/login")

(elemento campo_email entrada obrigatorio formato email seletor "#email")

(elemento campo_senha entrada obrigatorio formato senha minimo 8 seletor "#senha")

(elemento botao_entrar botao texto "Entrar" seletor "#entrar")

(elemento aviso_erro aviso visivel quando login_falhou))

(pagina catalogo

(rota "/produtos")

(elemento campo_busca entrada seletor "#busca")

(elemento botao_buscar botao texto "Buscar"))

(pagina produto

(rota "/produto")

(elemento botao_adicionar botao texto "Adicionar ao carrinho" seletor "#adicionar")

(elemento seletor_qtd selecao seletor "#quantidade"))

(pagina carrinho

(rota "/carrinho")

(elemento botao_finalizar botao texto "Finalizar compra" seletor "#finalizar")

(elemento botao_remover botao texto "Remover"))

(pagina checkout
 (rota "/checkout")
 (elemento campo_cartao entrada obrigatorio minimo 16 maximo 16 seletor "#cartao")
 (elemento campo_cvv entrada obrigatorio minimo 3 maximo 4 seletor "#cvv")
 (elemento campo_validade entrada obrigatorio formato data)
 (elemento botao_pagar botao texto "Pagar" seletor "#pagar")
 (elemento aviso_recusa aviso visivel quando pagamento_recusado))

(pagina confirmacao
 (rota "/confirmacao")
 (elemento numero_pedido entrada seletor "#pedido")
 (elemento link_continuar link texto "Continuar comprando"))

(pagina erro_pagamento
 (rota "/checkout/erro")
 (elemento botao_tentar botao texto "Tentar novamente"))

(inicio em inicio)

(final em confirmacao)

(transicao de inicio para login
 (via botao_login clicado))

(transicao de inicio para catalogo
 (via link_produtos clicado))

```
(transicao de login para catalogo
(via botao_entrar clicado)
(somente se
(e
(preenchido campo_email)
(preenchido campo_senha)))
(entao
(recebe sessao.autenticado verdadeiro)
(recebe pagamento_recusado falso)))
```

```
(transicao de login para login
(via botao_entrar clicado)
(somente se
(ou
(vazio campo_email)
(vazio campo_senha)))
(entao
(recebe login_falhou verdadeiro)))
```

```
(transicao de catalogo para produto
(via botao_buscar clicado))
```

```
(transicao de produto para carrinho
(via botao_adicionar clicado)
(somente se
(maior produto.estoque que 0))
(entao
(recebe carrinho.quantidade 1)))
```

```
(transicao de carrinho para checkout  
  (via botao_finalizar clicado)  
  (somente se  
    (e  
      (maior carrinho.quantidade que 0)  
      (igual sessao.autenticado a verdadeiro))))
```

```
(transicao de checkout para confirmacao  
  (via botao_pagar clicado)  
  (somente se  
    (e  
      (preenchido campo_cartao)  
      (preenchido campo_cvv)  
      (preenchido campo_validade)  
      (igual pagamento_recusado a falso)))  
  (entao  
    (recebe pedido.numero gerado)  
    (recebe carrinho.quantidade 0)))
```

```
(transicao de checkout para erro_pagamento  
  (via botao_pagar clicado)  
  (somente se  
    (igual pagamento_recusado a verdadeiro)))
```

```
(transicao de erro_pagamento para checkout  
  (via botao_tentar clicado)  
  (entao  
    (recebe pagamento_recusado falso)))
```

(transicao de confirmacao para inicio
(via link_continuar clicado))

(invariante checkout exige autenticacao
(sempre que
(estiver em checkout)
entao
(igual sessao.autenticado a verdadeiro)))

(invariante checkout exige carrinho nao vazio
(sempre que
(estiver em checkout)
entao
(maior carrinho.quantidade que 0)))

(invariante confirmacao exige pedido gerado
(sempre que
(estiver em confirmacao)
entao
(visivel numero_pedido)))

(dados usuario_valido
(email "user@loja.com")
(senha "Senha@123"))

(dados cartao_valido
 (numero "4111111111111111")
 (cvv "123")
 (validade "12/28"))

(dados cartao_invalido
 (numero "0000000000000000")
 (cvv "000")
 (validade "01/20"))

(fluxo compra completa
 (comecar em inicio)
 (passo clicar botao_login aguardar pagina login)
 (passo preencher campo_email com usuario_valido.email)
 (passo preencher campo_senha com usuario_valido.senha)
 (passo clicar botao_entrar aguardar pagina catalogo)
 (passo clicar botao_buscar aguardar pagina produto)
 (passo clicar botao_adicionar aguardar pagina carrinho)
 (passo clicar botao_finalizar aguardar pagina checkout)
 (passo preencher campo_cartao com cartao_valido.numero)
 (passo preencher campo_cvv com cartao_valido.cvv)
 (passo preencher campo_validade com cartao_valido.validade)
 (passo clicar botao_pagar aguardar pagina confirmacao)
 (terminar em confirmacao))

(fluxo pagamento recusado
 (comecar em checkout)
 (passo preencher campo_cartao com cartao_invalido.numero)
 (passo preencher campo_cvv com cartao_invalido.cvv)
 (passo preencher campo_validade com cartao_invalido.validade)
 (passo clicar botao_pagar aguardar pagina erro_pagamento)
 (terminar em erro_pagamento)))

Caso de uso: loja virtual completa

8

páginas

11

transições

3

invariantes

2

fluxos

```
inicio → login → catalogo → produto → carrinho  
      → checkout → confirmacao  
                ↳ erro_pagamento
```

Duas transições concorrentes mutuamente exclusivas: login (campos válidos/vazios) e checkout (pagamento aceito/recusado), ambas provadas automaticamente pela análise.

Saída de (print-analysis loja)

=====

Páginas: 8 | Transições: 11 | Invariantes: 3 | Fluxos: 2

Alcançabilidade:

- [ok] inicio
- [ok] login
- [ok] catalogo
- [ok] produto
- [ok] carrinho
- [ok] checkout
- [ok] confirmacao
- [ok] erro_pagamento

Condições concorrentes:

- [ok] login via botao_entrar -> catalogo e login via botao_entrar -> login são mutuamente exclusivas
- [ok] checkout via botao_pagar -> confirmacao e checkout via botao_pagar -> erro_pagamento são mutuamente exclusivas

Invariantes:

- [ok] checkout exige autenticacao
- [ok] checkout exige carrinho nao vazio
- [ok] confirmacao exige pedido gerado

Fluxos:

- [ok] compra completa — 11 passos
- [ok] pagamento recusado — 4 passos

Resultado: 0 erros | 2 avisos

- [aviso] produto -> carrinho via botao_adicionar: sem transição alternativa para produto.estoque igual a 0
- [aviso] carrinho -> checkout via botao_finalizar: sem transição alternativa para carrinho.quantidade igual a 0

Do modelo declarativo ao teste executável

```
(fluxo compra completa
  (começar em inicio)
  (passo clicar botao_login
    aguardar pagina login)
  (passo preencher campo_email
    com usuario_valido.email)
  ...
  (terminar em confirmacao))
```

generate-cypress



```
cy.visit('/');
cy.get('#botao_login').click();
cy.url().should('include', '/login');
cy.get('#email').type('user@loja.com');
// ...
cy.url().should('include', '/confirmacao');
```

Seletores, URLs e dados vêm das declarações do modelo — nada é escrito à mão no teste gerado.

Próximos passos possíveis

01

Invariantes com antecedentes arbitrários

Hoje só (estiver em pagina) é suportado; estender para preenchido, visível etc.

02

Backend Playwright

Gerar testes para Playwright além de Cypress, a partir do mesmo modelo.

03

Execução simbólica de fluxos

Validar se as guardas das transições seriam de fato satisfeitas durante o fluxo.

04

Visualização do grafo de navegação

Gerar diagrama (DOT/Graphviz) automaticamente a partir do wn-site.

Fontes consultadas

- FLATT, M.; FINDLER, R. B. The Racket Reference. PLT Inc., 2024. Disponível em: <https://docs.racket-lang.org/reference/>
- SANTANCHÈ, A. Material de apoio da disciplina Paradigmas de Programação, 2026.
- Documentação oficial do Cypress. Disponível em: <https://docs.cypress.io/>